

Pipeline de Observabilidad End-to-End con OpenTelemetry

Entregable: Reporte Técnico Cloud: AWS ECS Fargate + RDS PostgreSQL Fecha: Agosto 2026

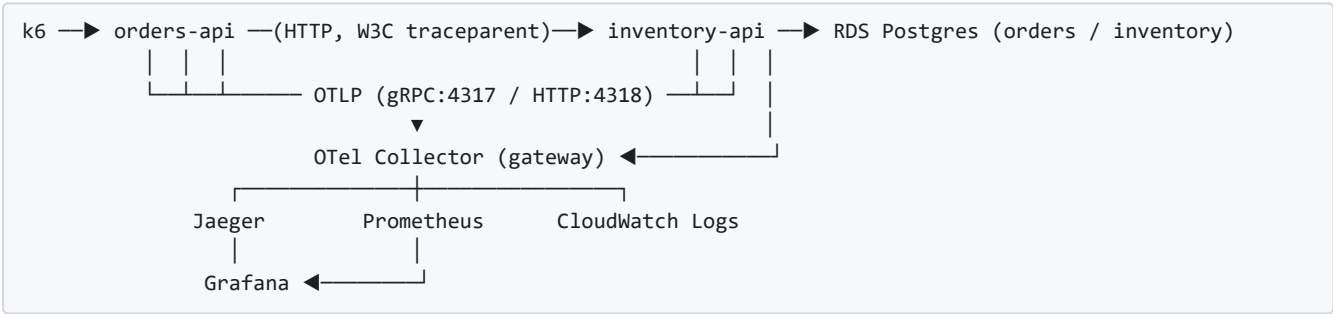
1. Resumen ejecutivo

Se implementó y validó un pipeline de observabilidad end-to-end basado en OpenTelemetry sobre dos microservicios Python/FastAPI (**orders-api** → **inventory-api**), emitiendo las tres señales —trazas, métricas y logs estructurados— hacia un OpenTelemetry Collector que las enruta a **Jaeger**, **Prometheus** y **CloudWatch Logs**, con visualización unificada en **Grafana**. El requisito crítico de **propagación W3C Trace Context** se verificó empíricamente: un mismo `trace_id` correlaciona logs, trazas y métricas a través de ambos servicios.

El despliegue objetivo es **AWS ECS Fargate + RDS PostgreSQL** mediante Terraform, con una estrategia de teardown efímero que respeta el free tier (~\$100). El benchmark de overhead (k6) demostró un costo de instrumentación medible: **-38% de throughput**, **+31% de latencia p99** y **+19-27% de memoria**, mitigable con muestreo de trazas y batching.

2. Arquitectura

La solución se compone de dos microservicios con dependencia HTTP, cada uno con acceso a su propia base de datos, y un plano de observabilidad desacoplado vía OTLP.



Componente	Rol	Puerto
orders-api (service-a)	Crea pedidos, llama a inventory por HTTP, persiste la orden	8080
inventory-api (service-b)	Valida y reserva stock (UPDATE atómico)	8081
OTel Collector	Recibe OTLP y enruta las 3 señales (batch + memory_limiter + resource)	4317/4318/8889
Jaeger all-in-one	Almacenamiento y visualización de trazas	16686
Prometheus	Métricas (scrape del collector :8889 y :8888)	9090
Grafana	Dashboard 6 paneles + correlación por trace_id	3000
CloudWatch Logs	Logs JSON (free tier 5 GB/mes)	—
RDS PostgreSQL	Bases orders e inventory (una por servicio)	5432

Flujo de una orden: el cliente hace `POST /orders` → **orders-api** valida la entrada (Pydantic) → llama por HTTP a **inventory-api** (`POST /products/{id}/reserve`) propagando el `traceparent` → **inventory-api** descuenta stock con un `UPDATE` atómico → **orders-api** inserta la orden y responde 201. Cada salto genera spans de auto-instrumentación más spans custom de negocio.

3. Instrumentación con OpenTelemetry

3.1 Auto-instrumentación

Ambos servicios se ejecutan bajo `opentelemetry-instrument` (CLI de `opentelemetry-distro`), que aplica instrumentación automática de **FastAPI** (spans server), **httpx** (spans client, con propagación W3C automática) y **SQLAlchemy** (spans connect/execute con atributos `db.statement`, `db.system`). Versiones pineadas: SDK 1.29.0, instrumentaciones 0.50b0.

3.2 Spans custom y métrica custom

Señal	Elemento	Servicio	Propósito
Trazas	<code>persist_order</code>	<code>orders-api</code>	Span de negocio sobre el INSERT de la orden
Trazas	<code>check_stock</code>	<code>inventory-api</code>	Span sobre la consulta de stock
Trazas	<code>reserve_stock</code>	<code>inventory-api</code>	Span sobre el descuento atómico de stock
Métricas	<code>db_operation_duration</code>	<code>ambos</code>	Histograma (ms) de latencia DB — necesario porque SQLAlchemy <code>async</code> no emite <code>db.client.operation.duration</code>

3.3 Logs estructurados

Logs en JSON (`structlog`) con un processor que inyecta `trace_id` y `span_id` desde el span activo:

```
{"order_id": "da65151f-...", "event": "order.created", "trace_id": "3d6cf90ef96226881054ba92f6594942", "span_id": "be787879b17fac07", "level": "info", "timestamp": "2026-08-17T02:07:05.350200Z"}
```

4. Verificación de la correlación cross-signal (W3C Trace Context)

El requisito crítico es que el `trace_id` sea consistente entre las tres señales. Se validó con un trace real (3d6cf90ef96226881054ba92f6594942):

- **Logs:** el JSON de `order.created` contiene el mismo `trace_id` (arriba).
- **Trazas:** Jaeger muestra el árbol completo con 15 spans; los spans de `inventory-api` son hijos del span `POST /orders` de `orders-api`, confirmando la propagación `traceparent` vía `httpx`.
- **Métricas:** las métricas se etiquetan por `service_name` gracias a `resource_to_telemetry_conversion: enabled` en el exporter Prometheus.

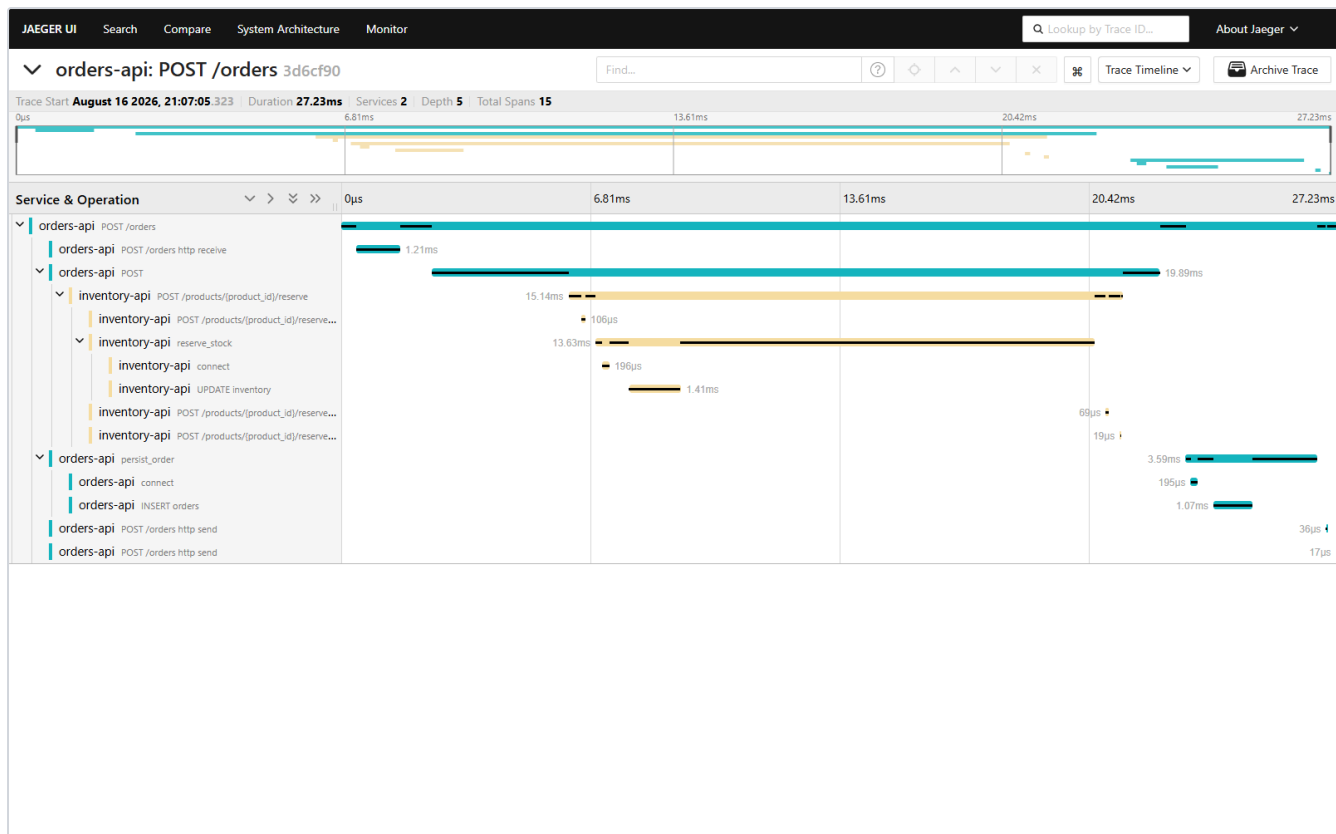


Figura 1 — Traza distribuida en Jaeger: spans de orders-api (FastAPI, httpx, SQLAlchemy, persist_order) y de inventory-api (reserve_stock), unificadas por un único trace_id (propagación W3C).

5. Backends y visualización

5.1 OTel Collector

Configuración con receiver OTLP (gRPC + HTTP), processors batch + memory_limiter + resource y exporters otel/jaeger (trazas), prometheus (:8889) y debug (logs en local; en AWS se sustituye por awsccloudwatchlogs). Nota: el exporter jaeger fue eliminado en el collector-contrib ≥ 0.109 , por lo que se usa otel hacia el receptor OTLP de Jaeger (COLLECTOR_OTLP_ENABLED=true).

5.2 Dashboard Grafana (6 paneles)

Panel	Señal	Consulta Prometheus
Latencia p99 HTTP	SLI	histogram_quantile(0.99, sum by (le, service_name) (rate(http_server_duration_milliseconds_bucket[1m])))
Error rate (5xx)	SLI	sum(rate(..._count{http_status_code=~"5.."}[1m])) / sum(rate(..._count[1m]))
Throughput (RPS)	SLI	sum by (service_name) (rate(http_server_duration_milliseconds_count[1m]))
Latencia DB p99	SLI	histogram_quantile(0.99, sum by (le, service_name) (rate(db_operation_duration_milliseconds_bucket[1m])))
CPU OTel Collector	Infra	rate(otelcol_process_cpu_seconds[1m])
Errores del Collector	Infra	sum by (exporter) (rate(otelcol_exporter_send_failed_spans[1m]))

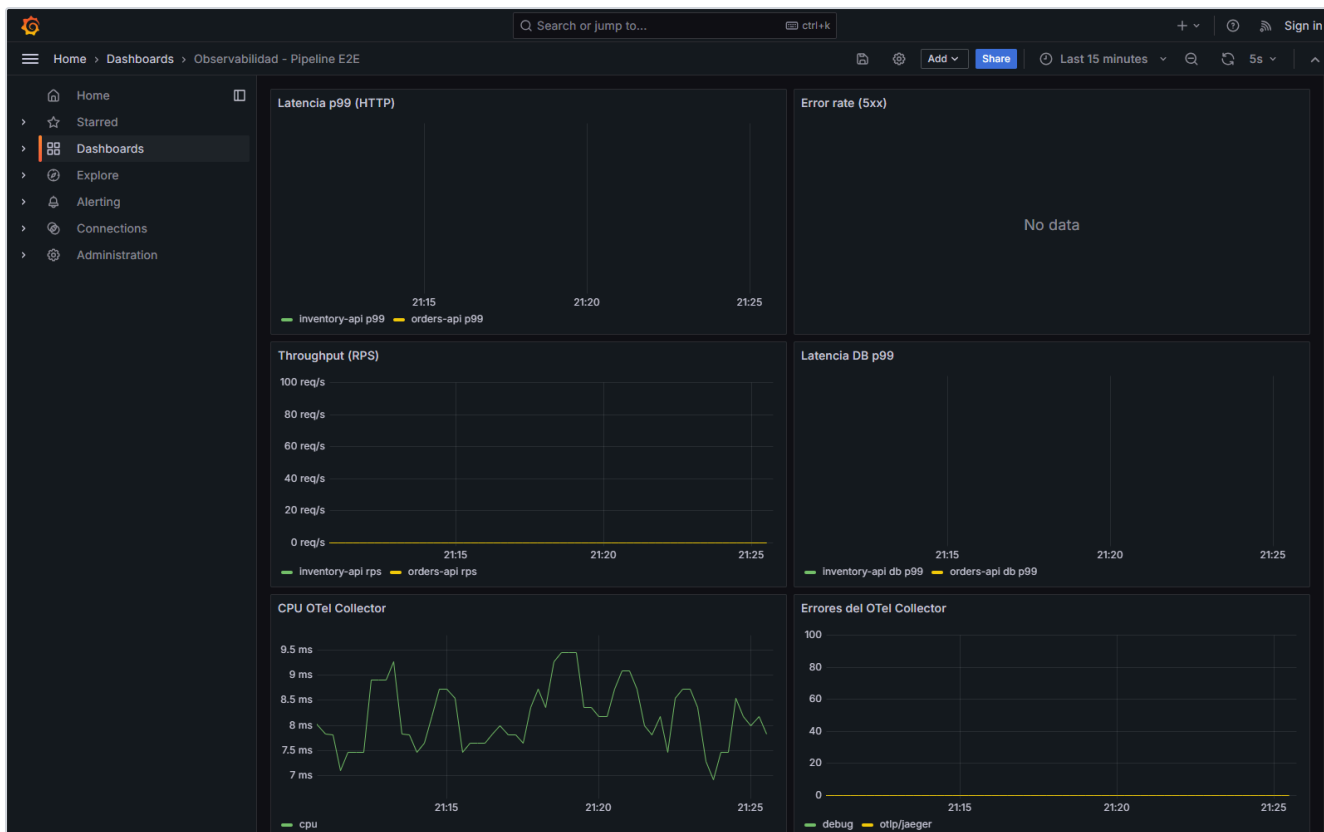


Figura 2 — Dashboard Grafana provisionado: 4 SLIs + CPU del collector + errores del collector, alimentado por Prometheus con datasource Jaeger para correlación.

6. Análisis de overhead (Fase 4 — k6)

6.1 Metodología

- **Con instrumentación:** servicios bajo opentelemetry-instrument (auto-instrumentación + spans custom + métricas OTLP cada 5s).
- **Sin instrumentación:** los mismos servicios con uvicorn directo (override de compose); el código de telemetría queda en no-op.
- **Carga:** k6, 50 VUs constantes sobre POST /orders, product_id rotado entre 100 productos para evitar contención de fila.

6.2 Resultados (local, Docker Desktop Windows)

Métrica	Sin OTEL	Con OTEL	Overhead
Throughput	217 req/s	135 req/s	−38%
Latencia promedio	229 ms	368 ms	+61%
Latencia p95	525 ms	900 ms	+71%
Latencia p99	945 ms	1240 ms	+31%
Memoria orders-api	72 MiB	86 MiB	+19%
Memoria inventory-api	64 MiB	81 MiB	+27%
CPU (1 worker)	~97%	~98%	similar (ver 6.3)

6.3 Interpretación

- El CPU se mantiene saturado en ambos casos (un único worker de uvicorn); el costo real de CPU se refleja en la **caída de throughput** a CPU comparable: más CPU por request.

- **Span fan-out alto:** cada `POST /orders` genera ~15–16 spans (~2000 spans/s a 135 req/s) exportados por OTLP.
- El export de métricas cada 5s (`OTEL_METRIC_EXPORT_INTERVAL=5000`) suma overhead de serialización bajo carga.
- **Sin muestreo** (default siempre-muestrea): con tail-based sampling (10–25%) el costo de export de trazas cae drásticamente.

7. Despliegue en AWS (ECS Fargate + RDS)

El despliegue se realiza con Terraform, con guardrails de coste para respetar el free tier (~\$100):

- **ECS Fargate** efímero: se levanta solo por sesión y se destruye con `terraform destroy`.
- **RDS PostgreSQL t4g.micro** single-AZ, `skip_final_snapshot`, teardown al terminar.
- **Sin NAT Gateway:** subnets públicas + VPC endpoint para ECR (evita ~\$32/mes).
- **Backends efímeros** (Jaeger/Prometheus/Grafana) sin volumen persistente; solo capturas como evidencia.
- Logs a **CloudWatch Logs** (free tier 5 GB/mes) en lugar de un contenedor Loki adicional.

8. Decisiones de diseño y lecciones aprendidas

Problema encontrado	Solución aplicada
Exporter jaeger eliminado en collector-contrib 0.109	Exporter otlp → receptor OTLP de Jaeger
Imagen del collector distroless (sin shell/wget)	Sin healthcheck; el exporter OTLP reintenta solo
Métricas sin label <code>service_name</code>	<code>resource_to_telemetry_conversion: enabled</code>
SQLAlchemy async no emite <code>db.client.operation.duration</code>	Histograma custom <code>db_operation_duration</code>
Métricas con lag de 60s	<code>OTEL_METRIC_EXPORT_INTERVAL=5000</code>
Contención de fila en el benchmark (todas las VUs sobre p1)	Repartir carga entre 100 productos — detectado vía trazas
Latencia HTTP elevada por conexión nueva por request	<code>httpx.AsyncClient</code> compartido (reutilización de conexiones)

9. Conclusiones

Se logró un pipeline de observabilidad completo y funcional con OpenTelemetry sobre microservicios FastAPI, cubriendo las tres señales y la correlación por `trace_id` exigida por el estándar W3C Trace Context. Las trazas distribuidas no solo cumplen el requisito de propagación, sino que demostraron su valor operativo al permitir aislar cuellos de botella (contención de fila y sobrecarga de conexiones) durante el benchmark. El overhead de instrumentación es medible y aceptable a baja carga; bajo carga alta, el span fan-out y la ausencia de muestreo lo amplifican, siendo el tail-based sampling la mitigación recomendada para producción. El despliegue en AWS ECS Fargate + RDS, con teardown efímero, mantiene el costo dentro del free tier.